INF10025 Data Management and Analytics

Task 3 – Pass Submission

Student Name: Truong Ngoc Gia Hieu
Student Number (VietNam/Australia): SWS01217/ 105565520

A. Task Pass3a

In the first section of *Task 3 (Pass Submission)*, students are required to open the ***MOVIEDB_SQL-1.txt*** file in **Notepad**. Then, students must replace all occurrences of ***XXXX*** with the ***last four digits*** of their ***StudentID***.

MOVIEDB_SQL-1 - Notepad

File Edit View

```
PROMPT  THIS SCRIPT CREATES AND POPULATES THE MOVIE DATABASE TABLES. Last updated Mar 2020.
PROMPT  Execute this script in ISQL JR.
PROMPT  When completed, do the following:
PROMPT  1. Clear the SQL statement box
PROMPT  2. Type this SQL statement and execute.   SELECT * FROM MOVIE5520
PROMPT     (This is how you see the rows in the Movie Table that script created)
PROMPT  3. Repeat similar SELECT statements for the ACTOR5520 and CASTING5520 tables.
PROMPT  .


PROMPT  Dropping any old versions of the Movie DB tables...

DROP TABLE CASTING5520;
DROP TABLE ACTOR5520;
DROP TABLE MOVIE5520;
DROP TABLE COLOURTYPE5520;
DROP TABLE RATING5520;

PROMPT ****
PROMPT **** Create RATING Table...
CREATE TABLE RATING5520 (
  RATINGCODE   VARCHAR(2)
, SHORTDESC    VARCHAR(30)
, LONGDESC     VARCHAR(100)
, PRIMARY KEY  (RATINGCODE)
);
PROMPT **** The RATING table has been created

PROMPT ****
PROMPT **** Create COLOURTYPE Table...
CREATE TABLE COLOURTYPE5520 (
  COLOURCODE   VARCHAR(1)
, COLOURNAME   VARCHAR(30)
, PRIMARY KEY  (COLOURCODE)
);
PROMPT **** The COLOURTYPE table has been created
```

Figure 1: MOVIEDB_SQL-1 Script in Notepad.

Next, students have to access the following website (*https://feenix-isqljr.swin.edu.au/*) and paste all the scripts in Notepad into the website.

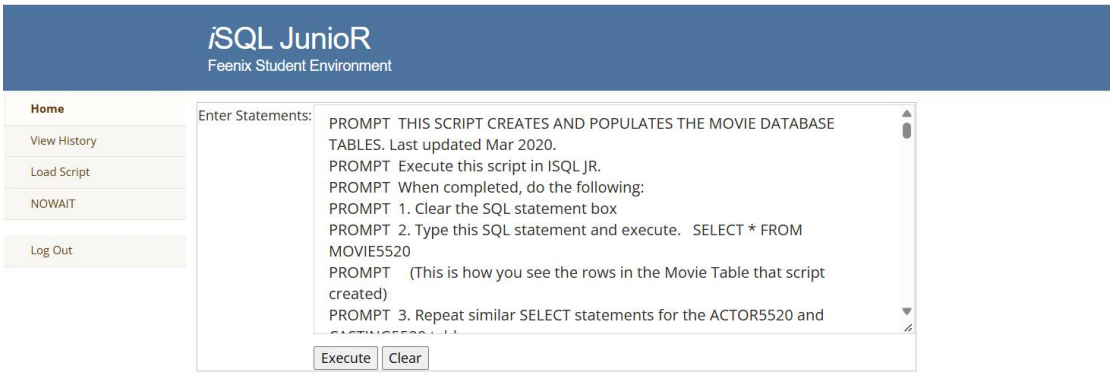


Figure 2: MOVIEDB_SQL-1 Script in iSQL JunioR.



Figure 3: Execution Result of MOVIEDB_SQL-1 Script in iSQL JunioR.

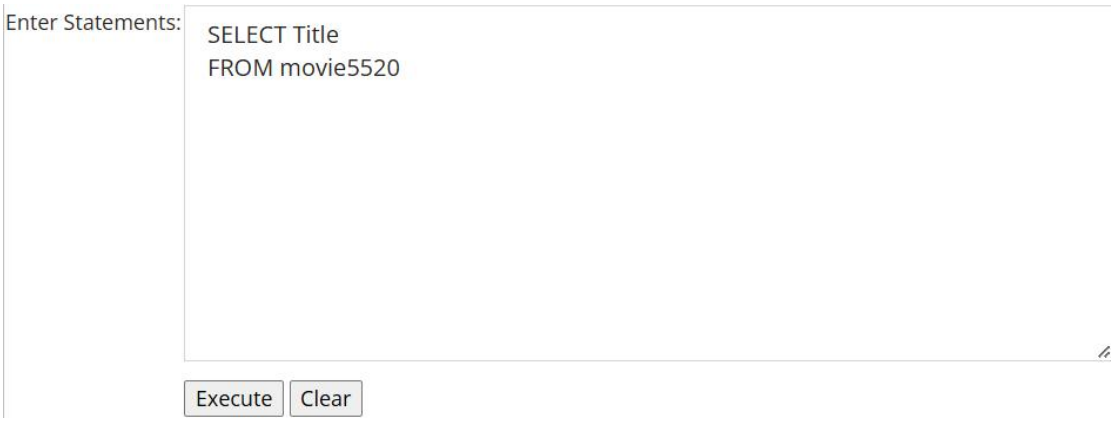
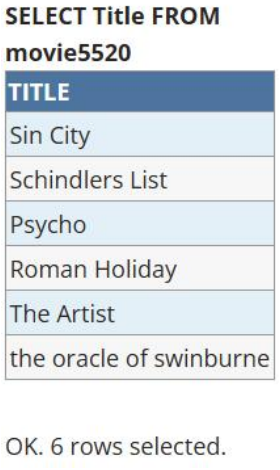


Figure 4: Entering SELECT Query for MOVIE5520 Table in iSQL JunioR.



SELECT Title FROM movie5520

TITLE
Sin City
Schindlers List
Psycho
Roman Holiday
The Artist
the oracle of swinburne

OK. 6 rows selected.

Figure 5: Execution Result of SELECT Query for movie5520 Table

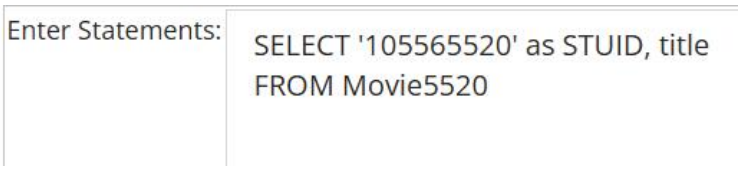


Figure 6: Entering SELECT Query with STUID Alias for Movie5520 Table.

SELECT '105565520' as STUID, title
FROM Movie5520

STUID	TITLE
105565520	Sin City
105565520	Schindlers List
105565520	Psycho
105565520	Roman Holiday
105565520	The Artist
105565520	the oracle of swinburne

OK. 6 rows selected.

Figure 7: Execution Result of SELECT Query with STUID Alias for Movie5520 Table

Subsequently, the lab instructions require students to write an SQL statement that retrieves only the ***StudID***, ***Movieno***, ***TITLE***, and ***RELYEAR*** columns for every row in the ***MOVIE5520*** table. Furthermore, the result must be sorted in descending order by Movieno. After drafting the code, students are instructed to save it in a ***Notepad*** file, paste it into ***iSQL JunioR***, and finally execute the statement.

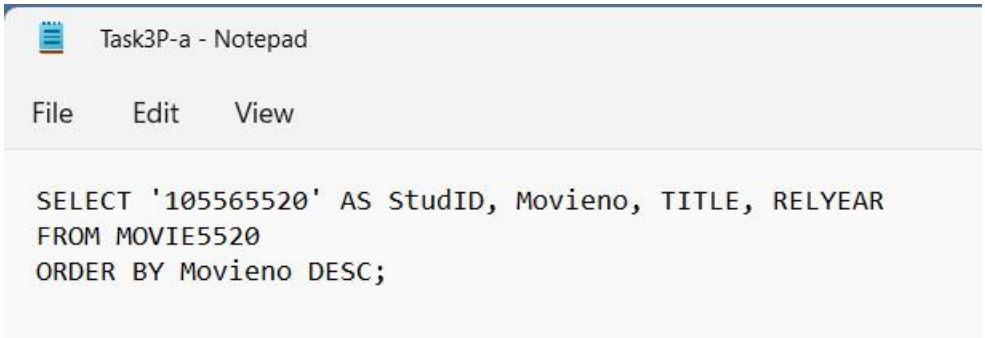


Figure 8: Task3Pa Script in Notepad.



Figure 9: Entering Task3Pa Query in iSQL JunioR.

STUID	MOVIENO	TITLE	RELYEAR
105565520	549053	Last Christmas	2019
105565520	512200	Jumanji: The Next Level	2019
105565520	489925	Eighth Grade	2018
105565520	475557	Joker	2019
105565520	466272	Once Upon a Time... in Hollywood	2019
105565520	459992	Long Shot	2019
105565520	458423	Mamma Mia! Here We Go Again	2018
105565520	454983	The Kissing Booth	2018
105565520	445571	Game Night	2018
105565520	433310	When We First Met	2018
105565520	419704	Ad Astra	2019
105565520	416477	The Big Sick	2017
105565520	402900	Oceans Eight	2018
105565520	401847	Can You Ever Forgive Me?	2018
105565520	400650	Mary Poppins Returns	2018

Figure 10: Execution Result of Task3Pa Query in iSQL JunioR.

B. Task Pass3b

In the second task of the third laboratory for the **Data Management and Analytics** course (INF10025), students are required to write and execute an SQL statement to list only the ***stuid, title***, ***relyear***, ***colourcode***, and ***ratingcode*** of every row in the movie table that has an 'M' rating. The results must be sorted in ascending order by relyear and title (note that two fields need to be sorted).

Enter Statements:

```
SELECT '105565520' AS StuID, Title, RELYEAR , Colourcode, ratingcode
FROM Movie5520
WHERE Ratingcode = 'M'
ORDER BY RELYEAR ASC, title ASC;
```

Figure 11: SQL Query Statement for Task Pass3b.

STUID	TITLE	RELYEAR	COLOURCODE	RATINGCODE
105565520	To Catch a Thief	1955	C	M
105565520	Charade	1963	C	M
105565520	Cool Hand Luke	1967	C	M
105565520	The Graduate	1967	C	M
105565520	Planet of the Apes	1968	C	M
105565520	The Odd Couple	1968	C	M
105565520	The Italian Job	1969	C	M
105565520	Catch-22	1970	C	M
105565520	M*A*S*H	1970	C	M
105565520	One Flew Over the Cuckoos Nest	1975	C	M

Figure 12: Output Result for Task Pass 3b SQL Query.

C. Task Pass3c

Subsequently, the third task asks students to write an SQL statement to list only the ***stuid***, ***movieno***, ***title***, and ***relyear*** of every row in the movie table that has either the title ***'Ocean's Eleven'*** or ***'Hamlet'***. Additionally, the list must be sorted in ***ascending*** order by ***title*** and ***relyear.***

Enter Statements:

```
SELECT '105565520' AS StuID, movieno, title, RELYEAR
FROM Movie5520
WHERE title='Oceans Eleven'
OR title='Hamlet'
ORDER BY RELYEAR ASC, title ASC;
```

Figure 13: SQL Query Statement for Task Pass3c.

STUID	MOVIENO	TITLE	RELYEAR
105565520	299	Oceans Eleven	1960
105565520	10264	Hamlet	1990
105565520	10549	Hamlet	1996
105565520	10688	Hamlet	2000
105565520	161	Oceans Eleven	2001

Figure 14: Output Result for Task Pass3c SQL Query.

D. Task Pass3d

Next, students are required to write and execute an SQL query to retrieve all columns for every movie in the movie table with a tmdb_score of 6.8 or higher and a release year of 2012 or earlier.

Furthermore, the results must be ordered by tmdb_score in descending sequence.

Enter Statements:

```
SELECT '105565520' AS StuID, MOVIE5520.*
FROM MOVIE5520
WHERE TMDB_SCORE >= 6.8 AND RELYEAR <= 2012
ORDER BY TMDB_SCORE DESC;
```

STUID	MOVIENO	TITLE	RELYEAR	RUNTIME	RATINGCODE	COLOURCODE	TMDB_SCORE	TMDB_VOTES	IMDB_ID
105565520	278	The Shawshank Redemption	1994	142	M	C	8.6	12151	tt0111161
105565520	424	Schindlers List	1993	195	M	B	8.5	7325	tt0108052
105565520	13	Forrest Gump	1994	142	PG	C	8.4	13591	tt0109830
105565520	539	Psycho	1960	109	MA	B	8.4	4466	tt0054215
105565520	680	Pulp Fiction	1994	154	MA	C	8.4	14055	tt0110912
105565520	550	Fight Club	1999	139	MA	C	8.4	15178	tt0137523
105565520	510	One Flew Over the Cuckoos Nest	1975	133	M	C	8.4	5073	tt0073486
105565520	155	The Dark Knight	2008	152	M	C	8.4	17733	tt0468569
105565520	122	The Lord of the Rings: The Return of the King	2003	201	M	C	8.4	12285	tt0167260
105565520	27205	Inception	2010	148	M	C	8.3	20761	tt1375666
105565520	274	The Silence of the Lambs	1991	119	MA	C	8.3	7754	tt0102926
105565520	120	The Lord of the Rings: The Fellowship of the Ring	2001	178	M	C	8.3	13495	tt0120737
105565520	807	Se7en	1995	127	MA	C	8.3	9718	tt0114369
105565520	77	Memento	2000	113	MA	C	8.2	7226	tt0209144
105565520	121	The Lord of the Rings: The Two Towers	2002	179	M	C	8.2	11649	tt0167261
105565520	629	The Usual Suspects	1995	106	MA	C	8.2	5255	tt0114814
105565520	1422	The Departed	2006	151	M	C	8.1	7239	tt0407887
105565520	9277	The Sting	1973	129	PG	C	8.1	1043	tt0070735
105565520	1366	Rocky	1976	120	PG	C	8.1	2381	tt0075148
105565520	857	Saving Private Ryan	1998	169	M	C	8.1	7785	tt0120815

Figure 15: Output Result for Task Pass3d SQL Query.

E. Task Pass3e

Next, the assignment requires writing and executing an SQL statement that uses the ***IN*** operator to filter movies with ***specific movieno values (78, 164, 1726, 2501, 57201***) from the Movie5520 table without using the OR operator. In addition, the statement retrieves the ***student ID (StuId)*** along with all columns from the table, and sorts the results in ***ascending order*** by ***movieno***.

Enter Statements:

```
SELECT '105565520' AS StuID, MOVIE5520.*
FROM MOVIE5520
WHERE movieno IN (78, 164, 1726, 2501, 57201)
ORDER BY movieno ASC;
```

Figure 16: SQL Query Statement for Task Pass3e.

STUID	MOVIENO	TITLE	RELYEAR	RUNTIME	RATINGCODE	COLOURCODE	TMDB_SCORE	TMDB_VOTES	IMDB_ID
105565520	78	Blade Runner	1982	117	MA	C	7.9	6986	tt0083658
105565520	164	Breakfast at Tiffany's	1961	115	PG	C	7.7	1986	tt0054698
105565520	1726	Iron Man	2008	126	PG	C	7.5	14150	tt0371746
105565520	2501	The Bourne Identity	2002	119	M	C	7.4	5088	tt0258463
105565520	57201	The Lone Ranger	2013	149	PG	C	6	3569	tt1210819

Figure 17: Output Result for Task Pass3e SQL Query.

F. Task Pass3f

Next, the assignment requires writing and executing an SQL statement to list all columns of the ***Actor table*** where the ***actor's password contains*** the *string 'ABCD'* regardless of case. As a result, the result set must be sorted in ***descending order by ActorNo.***

```
Enter Statements:
SELECT * FROM ACTOR5520
WHERE UPPER(password) LIKE '%ABCD%'
ORDER BY ActorNo DESC;
```

Figure 18: SQL Query Statement for Task Pass3f.

ACTORNO	FULLNAME	GIVENNAME	SURNAME	GENDER	BIRTHDATE	BIRTHCOUNTRY	PASSWORD	NOOFDOGS	NOOFCATS
129868	J.D. Evermore	J.D.	Evermore	M	05-NOV-68	USA	aBCDcC	1	2
105584	Bret McKenzie	Bret	McKenzie	M	29-JUN-76	New Zealand	aBCdbb	3	0
39187	Olivia Colman	Olivia	Colman	F	30-JAN-74	England	BAbcDa	0	3
23076	Robert Hardy	Robert	Hardy	M	29-OCT-25	England	cABCDb	1	1
4432	Frances Conroy	Frances	Conroy	F	13-NOV-53	USA	dAABCD	2	1
2478	Tommy Flanagan	Tommy	Flanagan	M	03-JUL-65	Scotland	abcDBc	1	0
2207	Lois Smith	Lois	Smith	F	03-NOV-30	USA	abCDCC	1	1
884	Steve Buscemi	Steve	Buscemi	M	13-DEC-57	USA	dABcdA	3	0

Figure 19: Output Result for Task Pass 3f SQL Query.

G. Task Pass3g

Then, the assignment requires writing and executing a single SQL statement to list the student ID along with all movie columns that meet specific rating and TMDB score criteria, constrained by a ***runtime of less than 134 minutes***. Specifically, the records must match either a **'PG' rating with a TMDB score greater than 7.6**, or ***an 'M' rating with a TMDB score of 4.1 or less***. Finally, the resulting dataset must be sorted in ascending order by movie number.

```
Enter Statements:
SELECT * FROM MOVIE5520
WHERE ((RATINGCODE='PG' AND TMDB_SCORE >= 7.5) OR
        (RATINGCODE='M' AND TMDB_SCORE <= 4.1))
AND RUNTIME < 134
ORDER BY MOVIENO ASC;
```

Figure 20: SQL Query Statement for Task Pass3g.

MOVIENO	TITLE	RELYEAR	RUNTIME	RATINGCODE	COLOURCODE	TMDB_SCORE	TMDB_VOTES	IMDB_ID
137	Groundhog Day	1993	101	PG	C	7.5	3713	tt0107048
162	Edward Scissorhands	1990	105	PG	C	7.6	6800	tt0099487
164	Breakfast at Tiffany's	1961	115	PG	C	7.7	1986	tt0054698
642	Butch Cassidy and the Sundance Kid	1969	111	PG	C	7.6	968	tt0064115
773	Little Miss Sunshine	2006	102	PG	C	7.6	3194	tt0449059
804	Roman Holiday	1953	119	PG	B	7.9	848	tt0046250
1366	Rocky	1976	120	PG	C	8.1	2381	tt0075148
1633	Fried Green Tomatoes	1991	130	PG	C	7.8	572	tt0101921
1726	Iron Man	2008	126	PG	C	7.5	14150	tt0371746
9277	The Sting	1973	129	PG	C	8.1	1043	tt0070735
9377	Ferris Buellers Day Off	1986	103	PG	C	7.6	2218	tt0091042
118340	Guardians of the Galaxy	2014	121	PG	C	8	4318	tt2015381
207703	Kingsman: The Secret Service	2015	129	PG	C	7.6	9670	tt2802144
313369	La La Land	2016	129	PG	C	7.9	9242	tt3783958
330457	Frozen II	2019	103	PG	C	8	2478	tt4520988

Figure 21: SQL Query Statement for Task Pass3g.

H. Task Pass3h

In the eighth task of the laboratory, the assignment requires writing and executing a single SQL statement that joins the ***MOVIE5520*** table with the ***RATING5520*** table to display the ***student ID, movie number, title, release year, TMDB score, rating code,*** and the ***corresponding long description***. Therefore, the query uses an inner join based on the matching rating codes between both tables. Finally, the resulting dataset is sorted in ascending order by movie number.

Enter Statements:

```
SELECT '105565520' AS StuID, M.MOVIENO, M.TITLE, M.RELYEAR,
M.TMDB_SCORE, M.RATINGCODE, R.LONGDESC
FROM MOVIE5520 M
JOIN RATING5520 R ON M.RATINGCODE = R.RATINGCODE
ORDER BY M.MOVIENO ASC;
```

Figure 22: SQL Query Statement for Task Pass3h.

STUID	MOVIENO	TITLE	RELYEAR	TMDB_SCORE	RATINGCODE	LONGDESC
105565520	13	Forrest Gump	1994	8.4	PG	Parental guidance recommended for persons under 15 years
105565520	22	Pirates of the Caribbean: The Curse of the Black Pearl	2003	7.7	PG	Parental guidance recommended for persons under 15 years
105565520	24	Kill Bill: Vol. 1	2003	7.9	MA	Persons under 15 years must be accompanied by a mature adult
105565520	70	Million Dollar Baby	2004	7.9	M	Recommended for mature audiences 15 years and over
105565520	75	Mars Attacks!	1996	6.2	M	Recommended for mature audiences 15 years and over

Figure 23: Output Result for Task Pass3h SQL Query.

I. Task Pass3i

In the final task, the assignment requires writing and executing a single SQL statement that joins the ***MOVIE5520*** table with the ***RATING5520*** table to display the ***movie title, movie number, release year, runtime, rating code, and the matching short description***. Furthermore, the query restricts the results to movies with ***a release year of 2012*** or ***greater and a runtime of 94 minutes or less.*** Finally, the resulting dataset is sorted in ascending order by movie number.

Enter Statements:

```
SELECT '105565520' AS StuID, M.TITLE, M.Movieno, M.RELYEAR, M.Runtime,
R.SHORTDESC
FROM MOVIE5520 M
INNER JOIN RATING5520 R ON M.RATINGCODE = R.RATINGCODE
WHERE M.RELYEAR >= 2012
AND M.RUNTIME <= 94
ORDER BY M.Movieno ASC;
```

Figure 24: SQL Query Statement for Task Pass3i.

STUID	TITLE	MOVIENO	RELYEAR	RUNTIME	SHORTDESC
105565520	Gravity	49047	2013	91	PARENTAL GUIDANCE
105565520	Hotel Transylvania	76492	2012	91	GENERAL
105565520	The Wedding Video	122843	2012	94	MATURE (15+)
105565520	What If	212716	2013	93	MATURE (15+)
105565520	Alexander and the Terrible, Horrible, No Good, Very Bad Day	218778	2014	81	GENERAL
105565520	Dior and I	261860	2014	90	MATURE (15+)
105565520	Eighth Grade	489925	2018	94	MATURE (15+)

Figure 25: Output Result for Task Pass3i SQL Query

Conclusion

In conclusion, the Pass part of Lab 3 successfully demonstrated the practical application of SQL queries in retrieving and filtering data from relational database tables. Through tasks ranging from basic selections to multi-table inner joins, key SQL concepts such as conditional logic, sorting, and primary-foreign key relationships were effectively implemented. Furthermore, all query results for the Pass tasks were fully validated against the specified schema criteria and documented in accordance with the required academic reporting standards